

TED UNIVERSITY

SENG-491

ANALYSIS REPORT

ATTENDAI

Vugar Gurbanov (vugar.gurbanov@tedu.edu.tr) (SENG)

Ahmet Arda Bağdatlı (ahmetarda.bagdatli@tedu.edu.tr) (SENG)

Nuray Aliyeva (nuray.aliyeva@tedu.edu.tr) (SENG)

1-Table Of Contents

1-Table Of Contents	2
2-Executive Summary	3
3-Overview	3
4-Software Requirements.....	3
4.1- Functional Requirements	3
4.2- Nonfunctional Requirements	4
4.3- Design constraints	5
5- System Models	6
5.1- Scenarios	6
Scenario 1:	6
Scenario 2:	7
Scenario 3:	8
5.2- Use Case Model.....	10
5.3- Class Diagram	11
5.4- Navigation Path	12
6- References	12

2-Executive Summary

AttendAI is an automated attendance management program that targets both instructors and students of TED University. Using facial recognition implementation with face embeddings, the system will take snapshots using the camera, identify current students, and store encrypted face embeddings to be checked later. AttendAI enhances attendance accuracy, instructor load, and offers a student correction process in case they feel that their attendance was not documented correctly. This report summarizes the functional/non-functional requirements, system constraints, use cases/scenarios, and the high-level system behavior that specify the architecture of AttendAI and its operation.

3-Overview

AttendAI is a system that automates the historically manual attendance system through the use of snapshots taken by cameras and face embedding comparisons. The system will have a database of embeddings that is encrypted and an in-memory cache that will be used to make quick comparisons during the running sessions. The teachers are able to set detection options like snapshot interval and confidence acts, and the pupils are able to see their attendance records and ask corrections in case needed.

AttendAI's primary goals are:

- Increasing attendance accuracy
- Reducing instructor workload
- Ensuring compliance with KVKK biometric data regulations
- Providing transparency and self-service tools for students

4-Software Requirements

4.1- Functional Requirements

Snapshot & Embedding Processing

- FR1 - The system shall capture snapshots using the selected webcam.
- FR2 - The system shall generate face embeddings from each snapshot.
- FR3 - The system shall compare new embeddings with stored embeddings and produce a confidence score.
- FR4 - The system shall check cached embeddings before performing database queries.
- FR5 - The system shall fetch embeddings from the database when cache comparisons fail.
- FR6 - The system shall store embeddings corresponding to students detected as present.

Instructor Configuration

- FR7 - The system shall allow instructors to configure snapshot frequency.
- FR8 - The system shall allow instructors to adjust the confidence threshold for recognition.

User Interactions

- FR9 - The system shall allow students to view attendance records and request corrections.
- FR10 - The system shall allow instructors to view, edit, and correct their attendance sessions.

4.2- Nonfunctional Requirements

Security & Privacy

- NFR1 - The system shall save only embeddings, not raw facial images.
- NFR2 - The system shall delete all snapshot images after processing.
- NFR3 - Passwords shall be hashed and salted using Argon2id.
- NFR4 - Embeddings shall be encrypted using AES-256 before storage.
- NFR5 - The system must comply with KVKK regulations.
- NFR6 - Students must be informed about biometric data usage and must provide consent.

Performance

NFR7 - The system shall process a classroom snapshot ([?]30 students) in under 1 minute.

NFR8 - The frontend must respond to user actions within 2 seconds.

Reliability & Logging

NFR9 - The system shall log all failures involving snapshot creation, account creation, or database operations.

NFR10 - The system shall generate daily database backups.

Access Control

NFR11 - The system shall enforce role-based access control for students, instructors, and administrators.

4.3- Design constraints

DC1 - The backend shall be implemented in Python.

DC2 - PostgreSQL shall be used as the database system.

DC3 - The frontend shall be implemented using Flask.

DC4 - InsightFace shall be used for face embedding generation.

5- System Models

5.1- Scenarios

Scenario 1:

Instructor Starts an Attendance Session

ID: UC-01

Actors: Instructor

Frequency: Once per lecture

Preconditions:

PRE1 - Instructor is logged in.

PRE2 - Instructor has configured snapshot settings or is using defaults.

PRE3 - Instructor has provided lecture information.

PRE4 - A webcam is available to the system.

Postconditions:

POS1 - A new attendance session entry is created.

POS2 - The frontend begins capturing snapshots.

Main Flow:

1 - Instructor clicks "Start Attendance Session".

2 - Frontend sends request to backend to create a new session.

3 - Backend creates session entry in the database.

- 4 - Backend returns confirmation.
- 5 - Frontend begins capturing snapshots.
- 6 - Instructor is notified that the session is active.

Exception Flow:

EX1: Database Entry Creation Fails

- 1 - Backend fails to create session entry and raises an error.
- 2 - Error is logged.
- 3 - Frontend receives error response.
- 4 - Instructor is informed and asked to retry.

Scenario 2:

Student Requests Attendance Correction

ID: UC-02

Actors: Student

Frequency: On-demand

Preconditions:

PRE1 - Student is logged in.

PRE2 - Student is viewing their attendance record for a valid class.

Postconditions:

POS1 - A correction message is stored in the database.

POS2 - Student receives confirmation of successful submission.

Main Flow:

- 1 - Student opens correction request form.
- 2 - Student fills in details and submits the form.
- 3 - Frontend sends data to backend.
- 4 - Backend stores a correction message linked to the session.

Exception Flow:**EX1:** Message Creation Fails

- 1 - Backend fails to create message entry and raises an error.
- 2 - Error is logged.
- 3 - Frontend displays error message to the student.

Scenario 3:

Instructor Edits Attendance

ID: UC-03

Actors: Instructor

Frequency: When corrections are needed

Preconditions:

PRE1 - Instructor is logged in.

PRE2 - Instructor is viewing a specific attendance session.

Postconditions:

POS1 - Attendance data is updated.

POS2 - Instructor is notified.

POS3 - Instructor is redirected to the session list.

Main Flow:

- 1 - Instructor edits student attendance entries.
- 2 - Frontend bundles changes to send to backend.
- 3 - Instructor clicks "Save Changes".
- 4 - Backend updates session data in the database.
- 5 - Backend logs the modifications.
- 6 - Confirmation is sent to frontend.
- 7 - Instructor is notified and redirected.

Exception Flow:

EX1: Update Fails

- 1 - Backend fails to update session.
- 2 - Error is logged.
- 3 - Frontend displays failure message.

5.2- Use Case Model

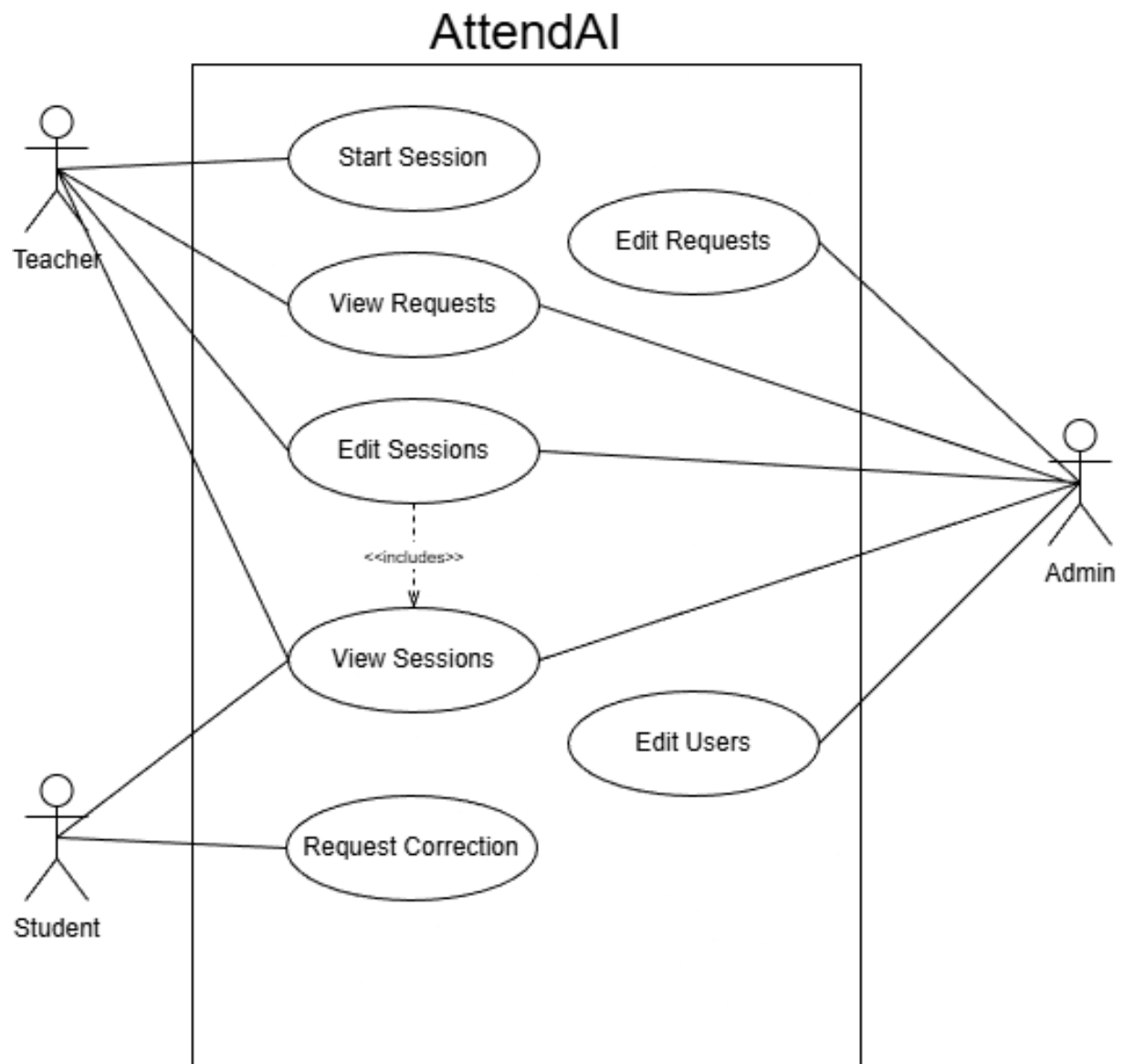


Diagram 1: Use Case Diagram for AttendAI

5.3- Class Diagram

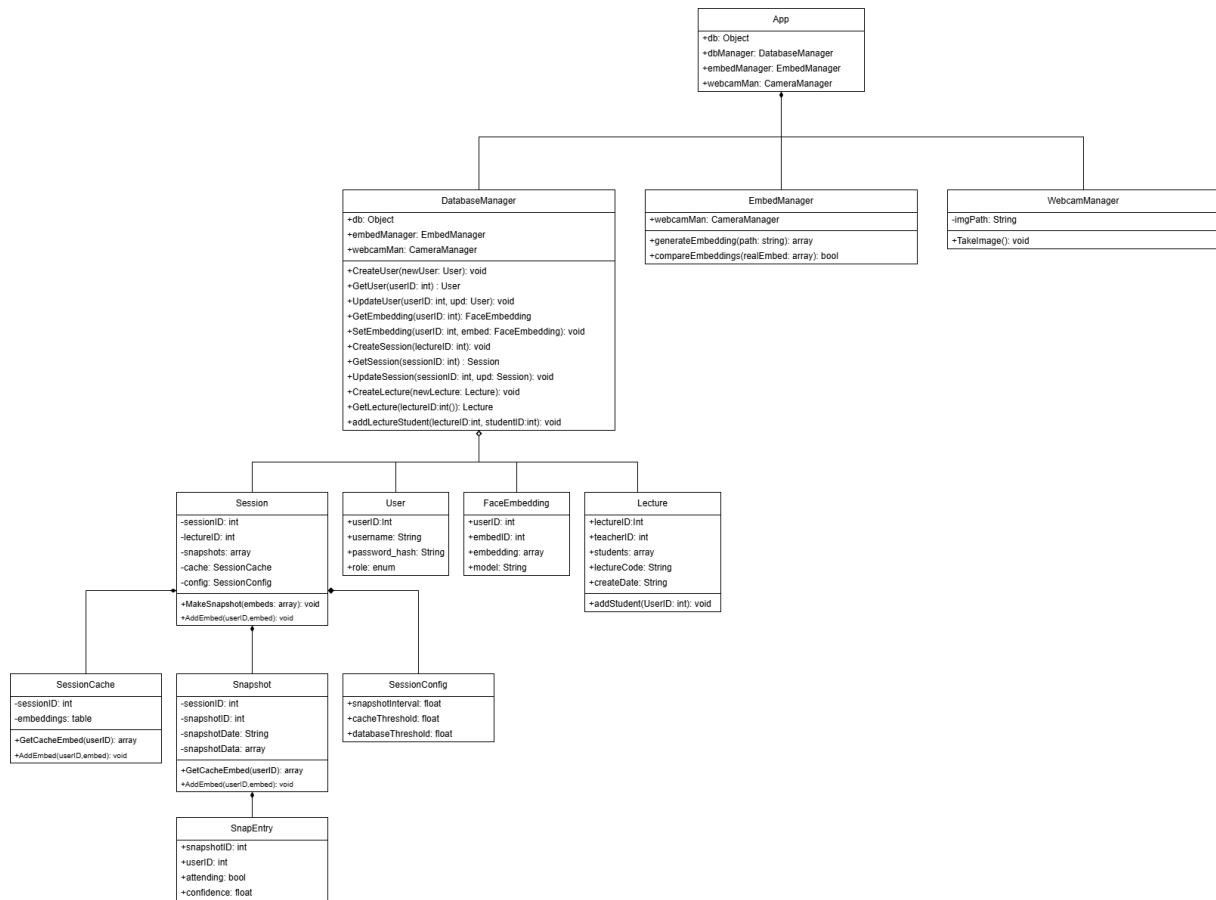


Diagram 2: Class Diagram for AttendAI

5.4- Navigation Path

This section illustrates basic flow:

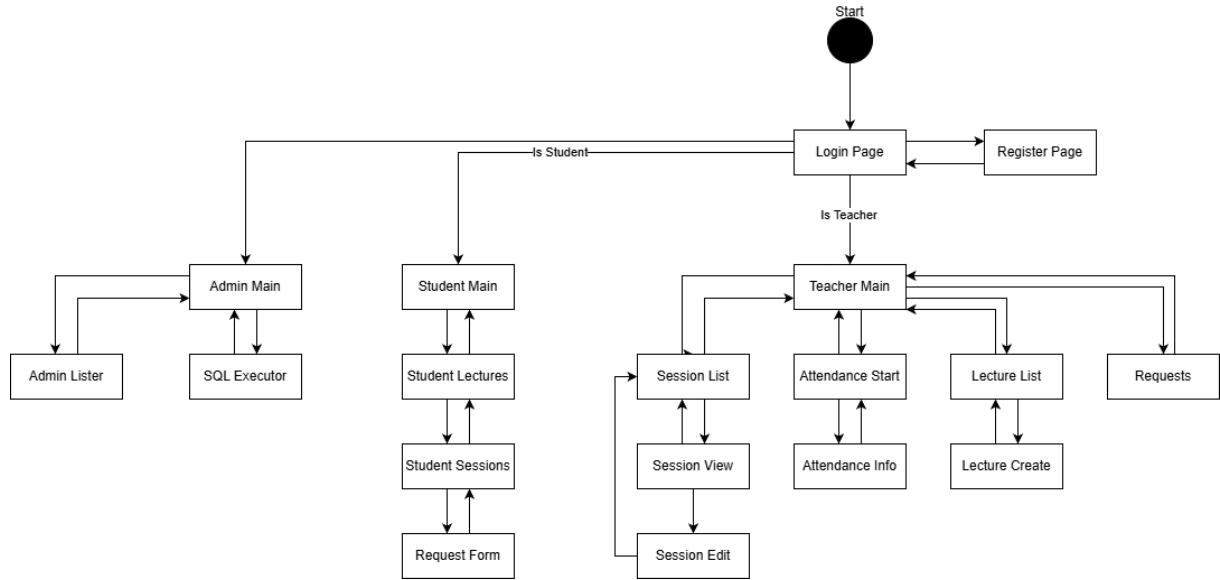


Diagram 3: Navigation Paths for AttendAI

6- References

- 1- Kişisel Verileri Koruma Kurumu. (n.d.). *KVKK resmî sitesi*. Retrieved November 30, 2025, from <https://www.kvkk.gov.tr/>
- 2- Python Software Foundation. (n.d.). *Python.org*. Retrieved November 30, 2025, from <https://www.python.org/>
- 3- PostgreSQL Global Development Group. (n.d.). *PostgreSQL: The world's most advanced open-source relational database*. Retrieved November 30, 2025, from <https://www.postgresql.org/>
- 4- Pallets Projects. (n.d.). *Flask (A Python micro-framework)*. Retrieved November 30, 2025, from <https://flask.palletsprojects.com/>
- 5- InsightFace community. (n.d.). *InsightFace: face recognition & analysis toolkit*. Retrieved November 30, 2025, from <https://www.insightface.ai/>